PRÁCTICA 05 - DETECCIÓN DE INTRUSIONES CON SURICATA

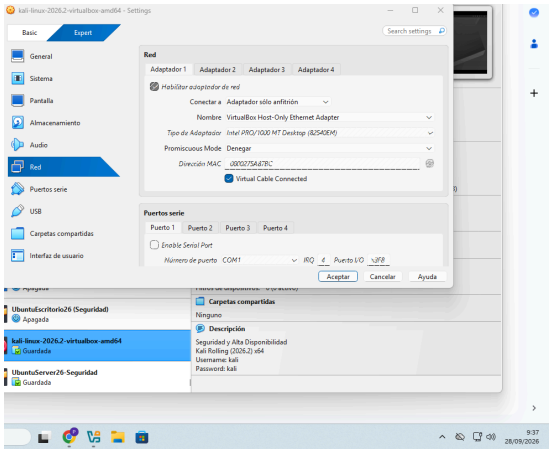
Seguridad y Alta Disponibilidad - Unidad 2

Pedro Hosannah - 2º ASIR - Curso 2026-2027

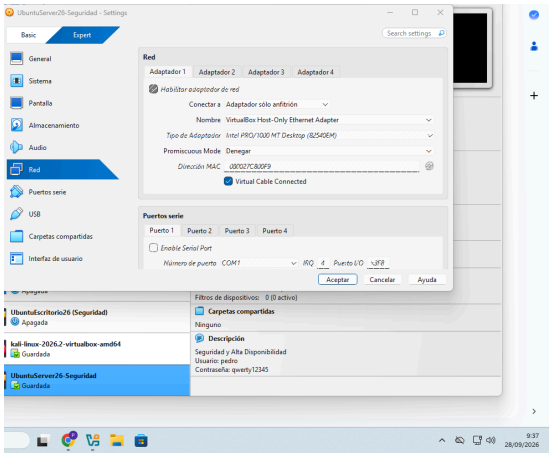
En esta práctica monté un IDS con Suricata en una red de laboratorio aislada. Utilicé una máquina Ubuntu Server como víctima y una máquina Kali Linux como atacante. Preparé la monitorización, lancé un ataque controlado desde Kali, creé dos reglas propias de detección y terminé aplicando una contramedida, siguiendo el enunciado de la Unidad 2.

1. Preparación del laboratorio

El enunciado pide dos máquinas virtuales en una red Host-Only para que puedan comunicarse entre ellas sin quedar expuestas a la red doméstica. He utilizado VirtualBox y he configurado el mismo adaptador solo-anfitrión en las dos máquinas.



Kali Linux configurada con Adaptador solo-anfitrión.

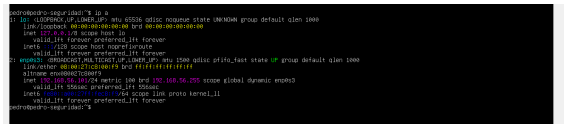


Ubuntu Server configurado con el mismo Adaptador solo-anfitrión.

1.1. Direccionamiento y conectividad

Máquina	Papel	Interfaz	IP real
UbuntuServer26-Seguridad	Víctima / IDS	enp0s3	192.168.56.101/24
Kali Linux 2026.2	Atacante	eth0	192.168.56.102/24

Las direcciones del PDF son de ejemplo. En mi laboratorio las máquinas recibieron estas IP, así que uso esos valores reales durante toda la práctica.



Ubuntu: enp0s3 = 192.168.56.101/24.

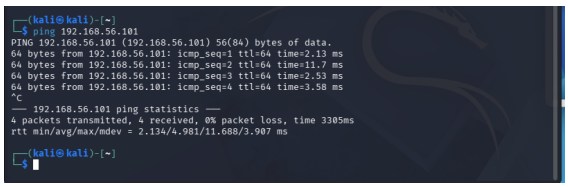


Kali: eth0 = 192.168.56.102/24.

Comprobé la comunicación en ambos sentidos con `ping`; las dos pruebas dieron 0 % de pérdida.



Ping de Ubuntu a Kali: 0 % de pérdida.



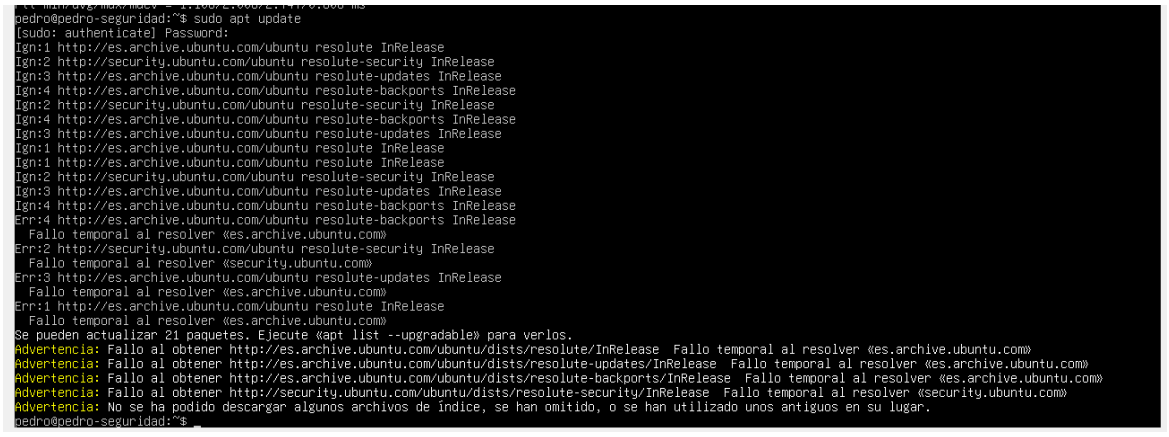
Ping de Kali a Ubuntu: 0 % de pérdida.

2. Paso 1 - Máquina víctima: instalación de herramientas

En Ubuntu Server tenía que instalar Suricata, vsftpd, tcpdump y jq. Antes de instalar los paquetes ejecuté `apt update`, pero apareció una incidencia debido al aislamiento de la red Host-Only.

2.1. Incidencia: la red Host-Only no tenía salida a Internet

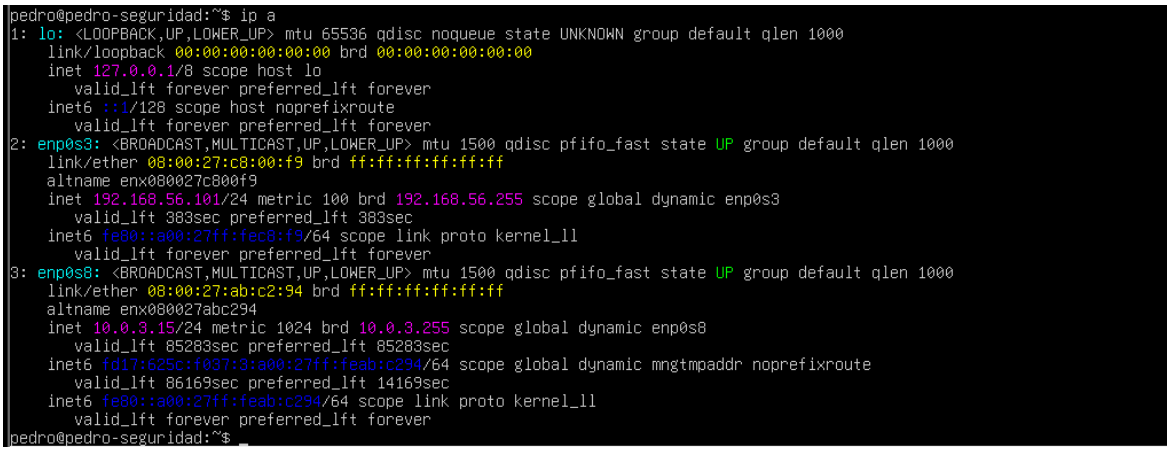
Al ejecutar la actualización de repositorios, Ubuntu no podía resolver los servidores de Ubuntu. El motivo era que el único adaptador activo era Host-Only, que está pensado para la red aislada del laboratorio.



`apt update` falla al no poder resolver los repositorios desde la red Host-Only.

Para poder descargar temporalmente los paquetes añadí un segundo adaptador NAT a Ubuntu. No sustituí la interfaz Host-Only: la mantuve como red de la práctica y utilicé NAT únicamente para las descargas.

```
enp0s3  -> Host-Only -> 192.168.56.101/24
enp0s8  -> NAT       -> 10.0.3.15/24
```



Ubuntu con la interfaz Host-Only enp0s3 y la interfaz NAT temporal enp0s8.

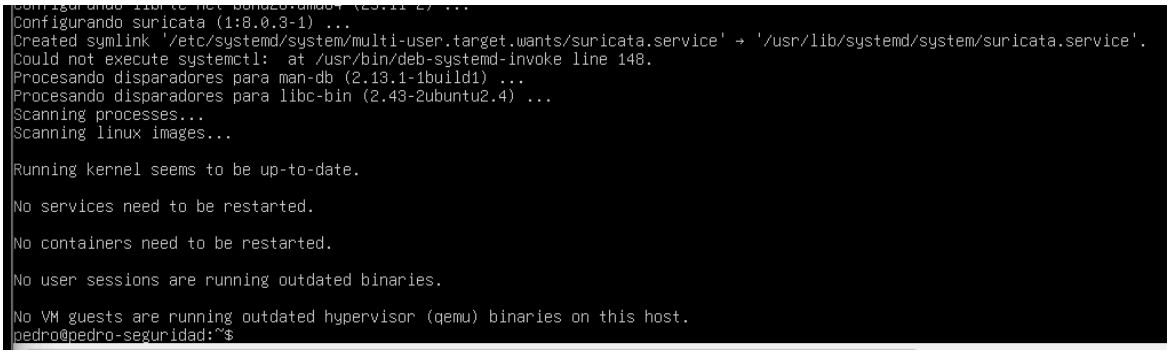


`sudo apt update` termina correctamente después de añadir NAT de forma temporal.

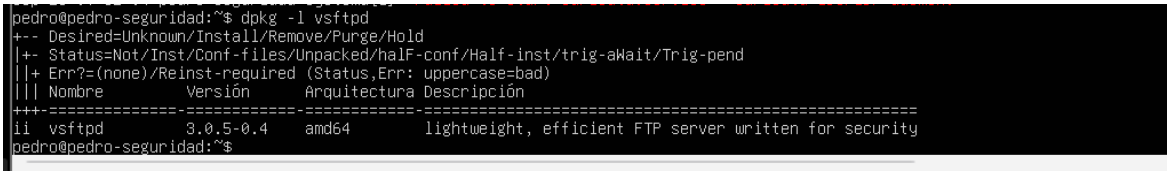
2.2. Instalación y comprobación de los paquetes

```
sudo apt install -y suricata vsftpd tcpdump jq
```

La instalación terminó correctamente. Después comprobé los paquetes: Suricata, tcpdump y jq estaban instalados. En `vsftpd` tuve un error al escribir el nombre y, al corregirlo, confirmé que también estaba instalado.



Final de la instalación de las herramientas de la práctica.

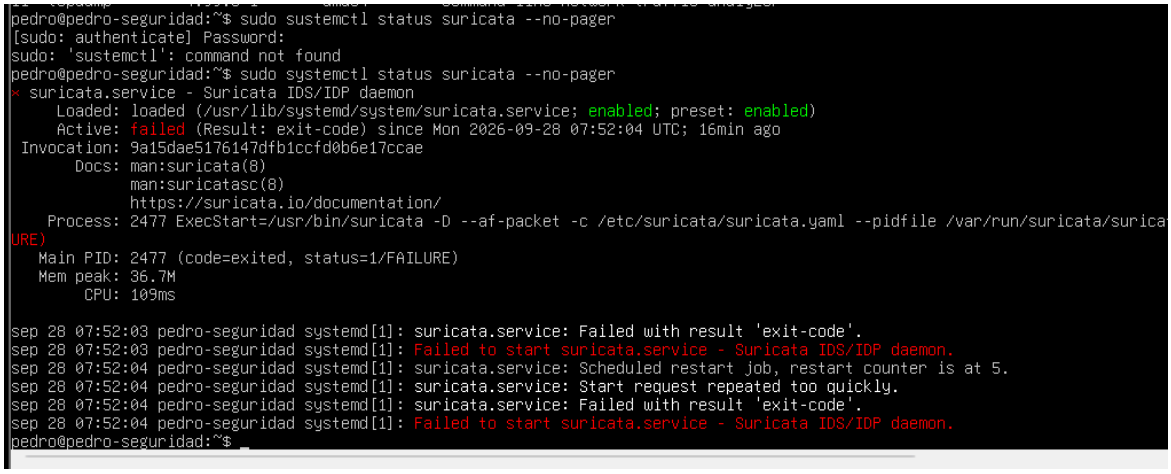


Comprobación de vsftpd instalado correctamente.

2.3. Incidencia: Suricata aparecía en estado failed

Antes de terminar la configuración comprobé el servicio de Suricata y aparecía como `failed`. No lo reinicié a ciegas. El enunciado indica que primero hay que configurar HOME_NET, la interfaz y las reglas, y

después validar con `suricata -T`. Por eso continué con la configuración antes de intentar arrancarlo de nuevo.

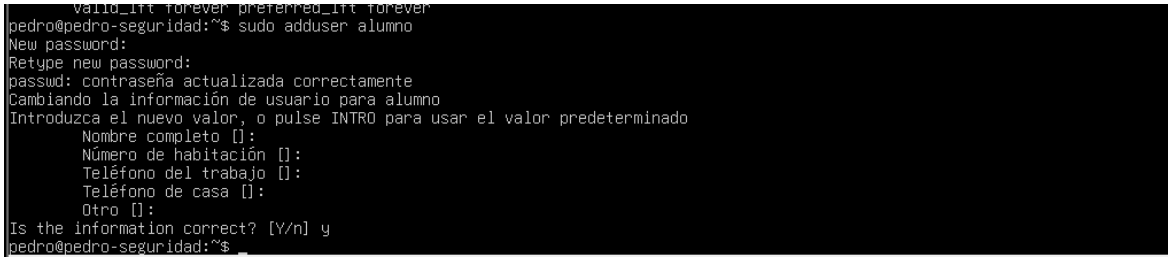


Estado inicial de Suricata: failed antes de completar la configuración.

2.4. Usuario para la prueba FTP

Creé el usuario `alumno` para usarlo más adelante en la prueba FTP desde Kali y comprobar si las credenciales se podían ver en texto claro.

```
sudo adduser alumno
```



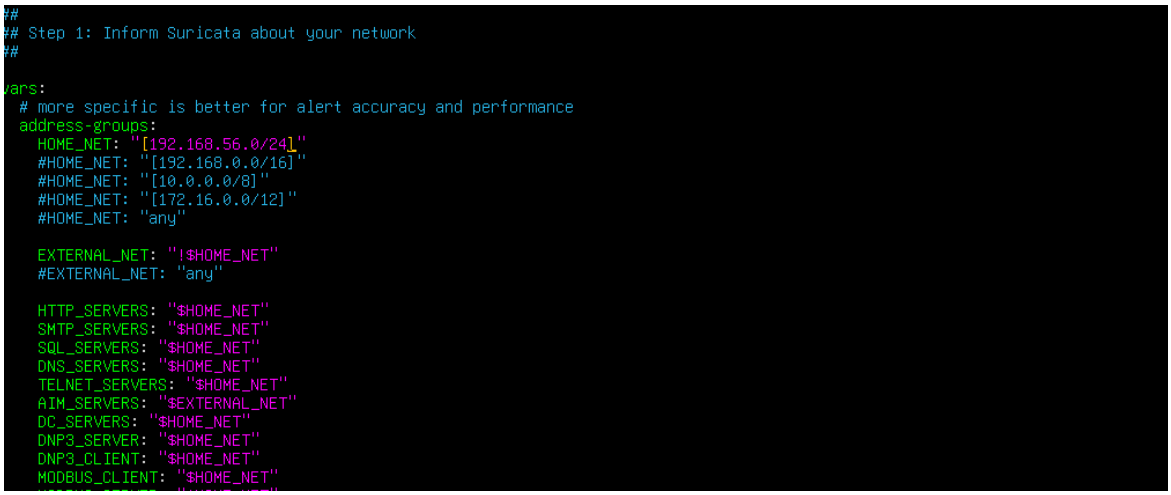
Creación del usuario alumno para la futura prueba FTP.

3. Paso 2 - Configuración de Suricata

En `/etc/suricata/suricata.yaml` configuré la red protegida, la interfaz de captura y los ficheros de reglas.

3.1. HOME_NET

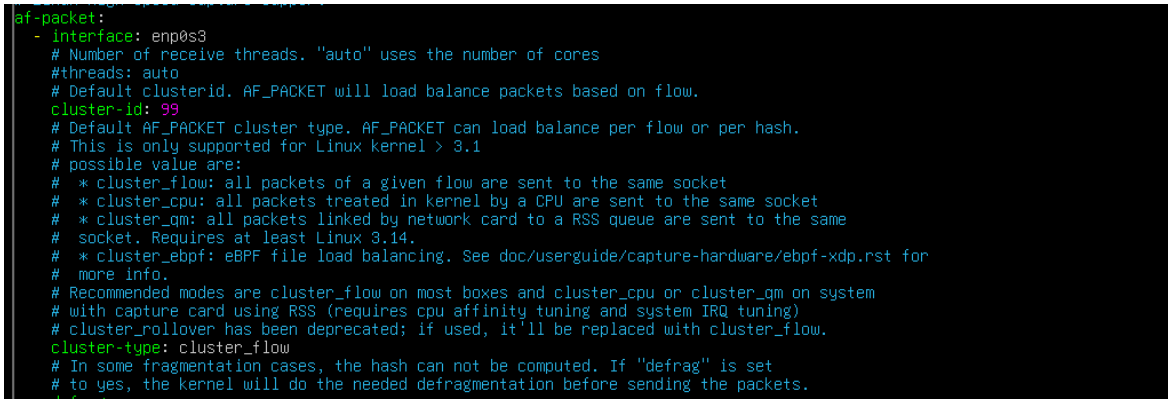
Configuré HOME_NET con la red real del laboratorio: `192.168.56.0/24`. Las otras líneas HOME_NET del fichero son ejemplos comentados y se dejaron sin activar.



HOME_NET configurado como 192.168.56.0/24.

3.2. Interfaz de captura

En `af-packet` cambié la interfaz que venía por defecto por `enp0s3`, porque es la interfaz Host-Only por la que circula el tráfico entre Ubuntu y Kali. La interfaz NAT temporal no se utilizará para la detección.



Suricata configurado para escuchar por enp0s3.

3.3. Ficheros de reglas

Añadí `local.rules` junto a `suricata.rules`. El primero se utilizará para las dos reglas propias de la práctica y el segundo contiene las reglas públicas descargadas con `suricata-update`.

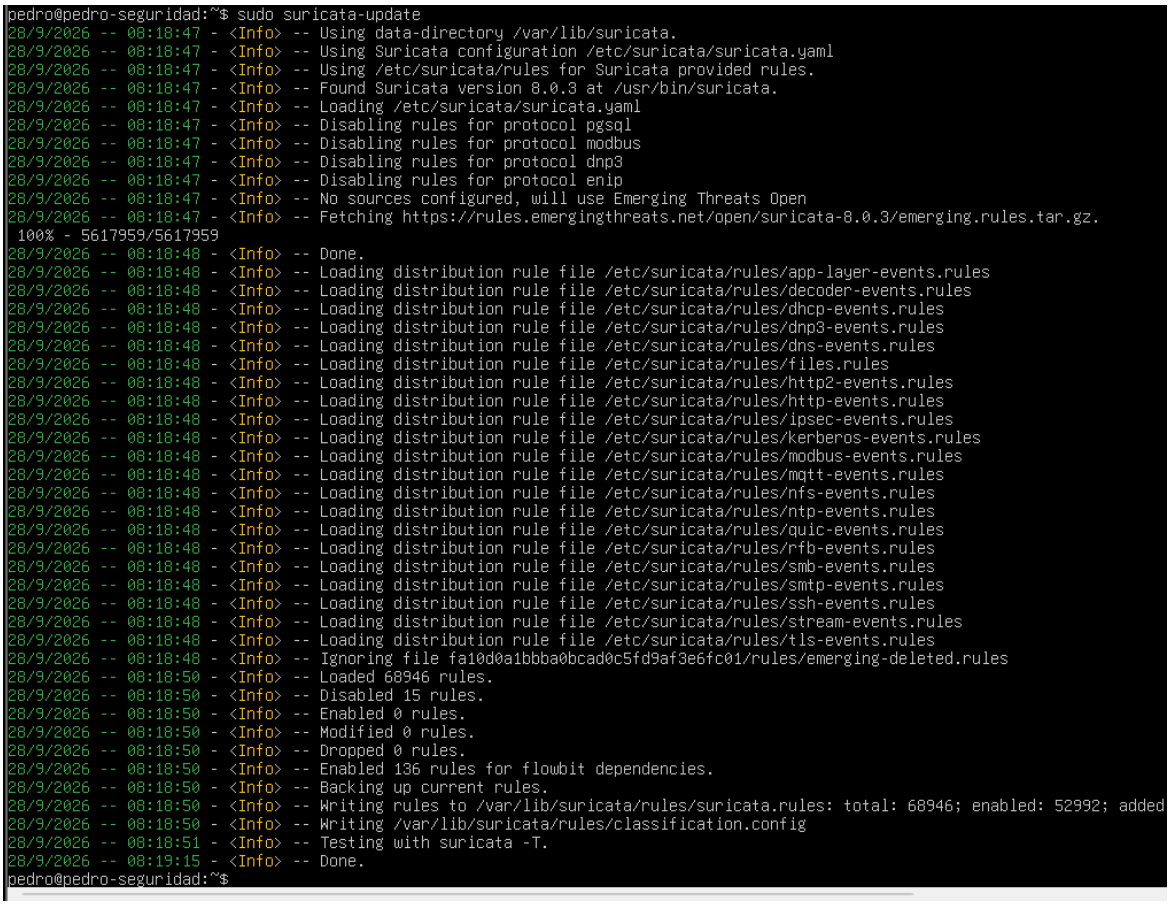


rule-files con suricata.rules y local.rules.

3.4. Descarga de reglas públicas

```
sudo suricata-update
```

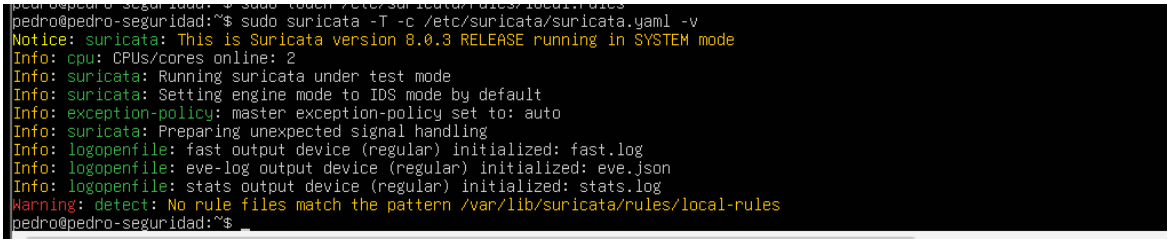
La descarga terminó correctamente. La herramienta obtuvo las reglas de Emerging Threats Open y generó `suricata.rules`. Al final realizó una prueba automática con Suricata.



`suricata-update` finaliza con `Done.` después de cargar las reglas públicas.

4. Incidencia y resolución: ruta de local.rules

Después de crear `local.rules` y ejecutar la validación manual apareció el aviso `No rule files match the pattern /var/lib/suricata/rules/local.rules`. La práctica indica `/etc/suricata/rules`, pero la instalación real de Suricata 8.0.3 está usando `/var/lib/suricata/rules` como `default-rule-path`.



Primera validación: Suricata avisa de que no encuentra local.rules en la ruta activa.

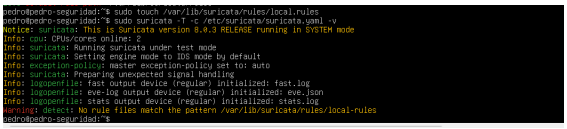
Lo comprobé directamente en el fichero de configuración con `grep`. Después creé el archivo también en `/var/lib/suricata/rules/local.rules`.

```
grep -n "default-rule-path" /etc/suricata/suricata.yaml
sudo touch /var/lib/suricata/rules/local.rules
```

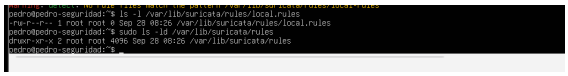


La ruta activa de reglas es /var/lib/suricata/rules.

Aun así, la validación seguía mostrando el mismo aviso. Comprobé que el archivo existía, pertenecía a root y tenía permisos de lectura. Paré en ese punto para revisar la causa antes de seguir cambiando la configuración.



El aviso continúa tras crear el archivo en la ruta activa.



local.rules existe y sus permisos son correctos.

4.1. Resolución de la incidencia

La causa no estaba en los permisos del fichero, sino en cómo Suricata estaba resolviendo la ruta de las reglas. Dejé `default-rule-path` en `/var/lib/suricata/rules` para las reglas descargadas y cambié la entrada de `local.rules` para usar su ruta absoluta: `/etc/suricata/rules/local.rules`.

Repetí la validación y esta vez Suricata procesó los dos ficheros de reglas y cargó 52.992 reglas sin fallos. El aviso de `local.rules` ya no apareció.

```
sudo suricata -T -c /etc/suricata/suricata.yaml -v
```

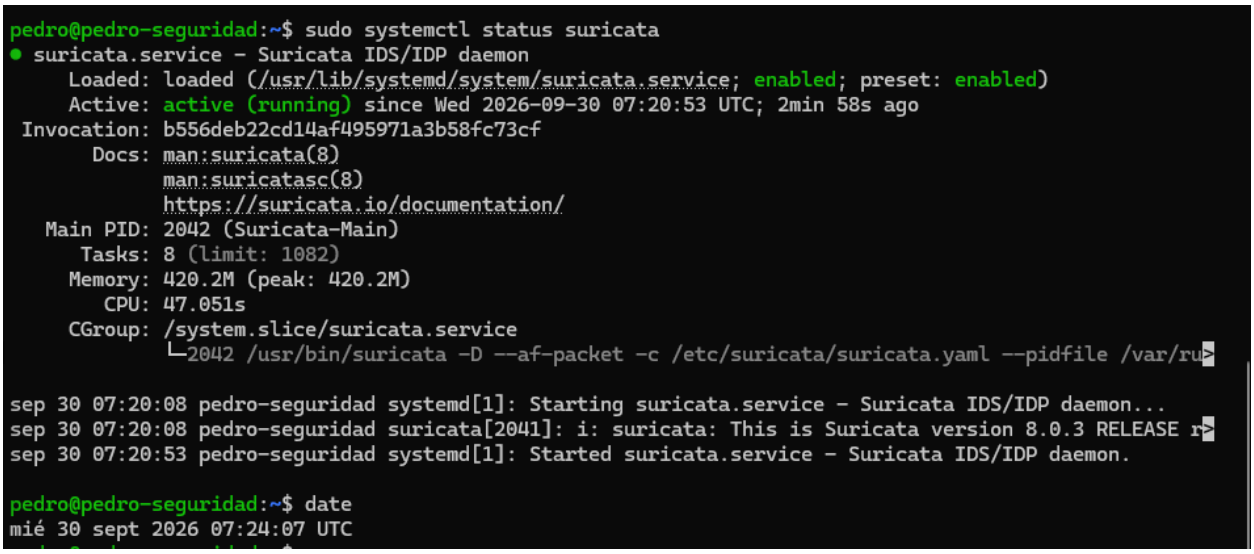
```
pedro@pedro-seguridad:~$ sudo suricata -T -c /etc/suricata/suricata.yaml -v
Notice: suricata: This is Suricata version 8.0.3 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 2
Info: suricata: Running suricata under test mode
Info: suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: suricata: Preparing unexpected signal handling
Info: logopenfile: fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Info: detect: 2 rule files processed. 52992 rules successfully loaded, 0 rules failed, 0 rules skipped
Info: threshold-config: Threshold config parsed: 0 rule(s) found
Info: detect: 52997 signatures processed. 1235 are IP-only rules, 4534 are inspecting packet payload,
46992 inspect application layer, 110 are decoder event only
Notice: mpm-hs: Rule group caching - loaded: 118 newly cached: 0 total cacheable: 118
Notice: suricata: Configuration provided was successfully loaded. Exiting.
pedro@pedro-seguridad:~$
```

Validación correcta de Suricata después de corregir la ruta de local.rules.

Reinicié el servicio y comprobé que Suricata quedaba activo.

```
sudo systemctl restart suricata
```

```
sudo systemctl status suricata
```

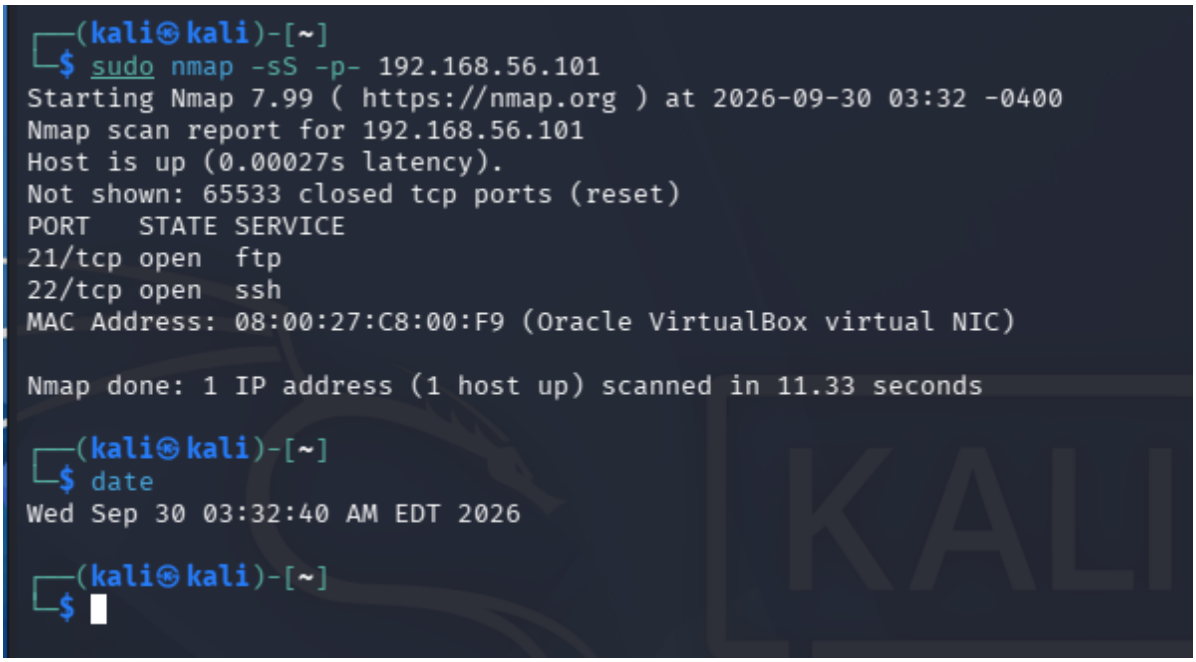


Suricata en estado active (running) después de resolver la incidencia.

5. Paso 3 - Atacar y observar

Con Suricata funcionando dejé `fast.log` abierto en Ubuntu y lancé desde Kali los dos escaneos indicados en el enunciado. El primero fue un SYN scan sobre todos los puertos TCP.

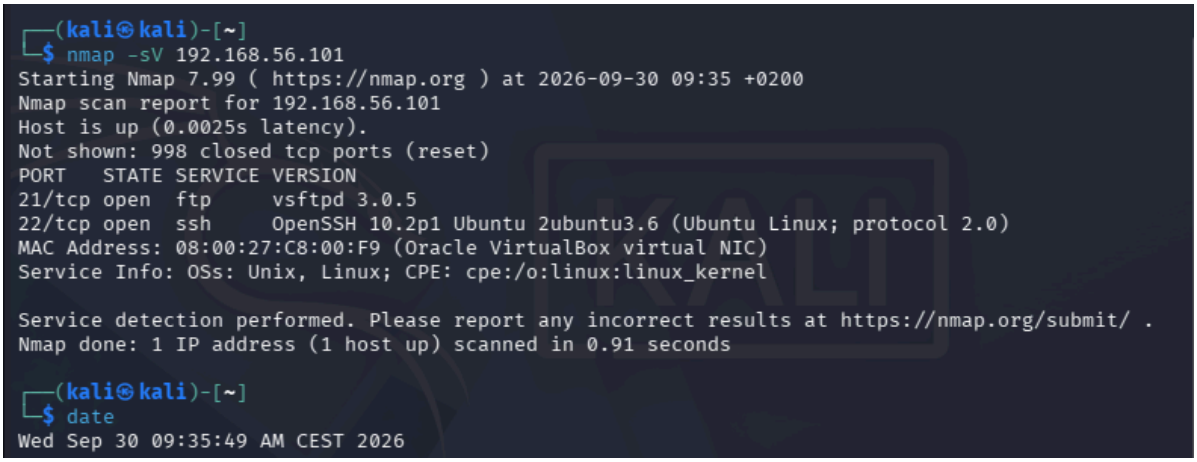
```
sudo nmap -sS -p- 192.168.56.101
```



Primer escaneo SYN completo desde Kali. En ese momento Kali todavía mostraba el huso EDT; después corregí la zona horaria y activé NTP.

El escaneo encontró abiertos los puertos 21/tcp y 22/tcp. Después utilicé `-sV` para identificar los servicios y sus versiones.

```
nmap -sV 192.168.56.101
```



Nmap identifica FTP con vsftpd 3.0.5 y SSH con OpenSSH 10.2p1 antes de aplicar la contramedida.

En `fast.log` no apareció una alerta clara que identificara este reconocimiento como escaneo SYN con las reglas públicas. Sí aparecieron otras alertas, como una firma informativa relacionada con Kali y DHCP, pero no las interpreté como detección del Nmap. Como no vi una detección clara del escaneo, pasé a comprobar las reglas propias del siguiente paso.

6. Paso 4 - Reglas propias de detección

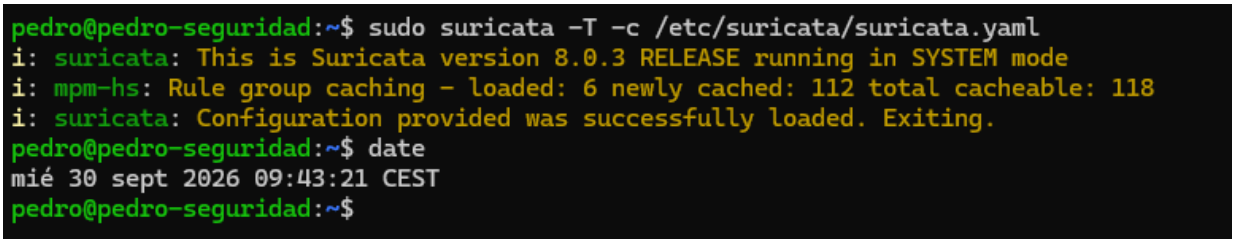
Añadí dos reglas a `/etc/suricata/rules/local.rules`. La primera detecta muchos paquetes SYN desde un mismo origen y la segunda busca el comando `PASS ` de FTP hacia el puerto 21.

```
alert tcp any any -> $HOME_NET any (msg:"UT2 Posible escaneo SYN"; flags:S; threshold:type
both, track by_src, count 20, seconds 5; classtype:attempted-recon; sid:1000001; rev:1;)
```

```
alert tcp any any -> $HOME_NET 21 (msg:"UT2 Credenciales FTP en claro";
flow:to_server,established; content:"PASS "; nocase; classtype:policy-violation;
sid:1000002; rev:1;)
```

Antes de reiniciar Suricata ejecuté otra vez el modo de prueba. Como la configuración cargó sin errores, reinicié el servicio para usar las reglas nuevas.

```
sudo suricata -T -c /etc/suricata/suricata.yaml
```



Validación correcta de la configuración después de añadir las dos reglas propias.

7. Paso 5 - Comprobar la detección

7.1. Regla de escaneo SYN

Repetí el escaneo SYN completo desde Kali para comprobar la Regla 1. El objetivo ya no era descubrir los puertos, sino generar el patrón de muchos SYN en pocos segundos que había definido en la regla.

```
sudo nmap -sS -p- 192.168.56.101
```

```
┌──(kali㉿kali)-[~]
└─$ date
Wed Sep 30 09:45:20 AM CEST 2026

┌──(kali㉿kali)-[~]
└─$ sudo nmap -sS -p- 192.168.56.101
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-30 09:45 +0200
Nmap scan report for 192.168.56.101
Host is up (0.00042s latency).
Not shown: 65533 closed tcp ports (reset)
PORT   STATE SERVICE
21/tcp open  ftp
22/tcp open  ssh
MAC Address: 08:00:27:C8:00:F9 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 11.32 seconds
```

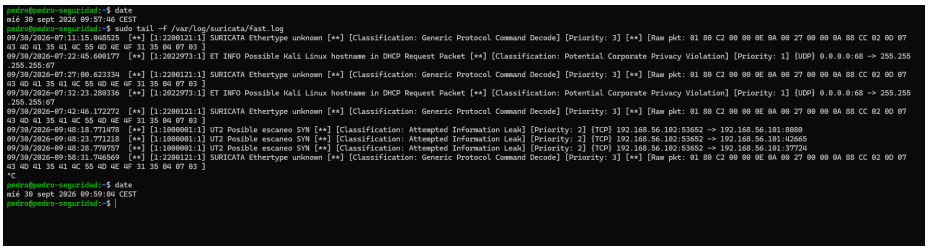
Repetición del escaneo SYN desde Kali para disparar la Regla 1.

Suricata registró varias alertas `UT2 Posible escaneo SYN` con SID 1000001. El origen era Kali (`192.168.56.102`) y el destino Ubuntu (`192.168.56.101`).

7.2. Regla de credenciales FTP en claro

Después probé la segunda regla. Desde Kali me conecté por FTP a Ubuntu con el usuario `alumno`. El servidor aceptó el inicio de sesión correctamente.

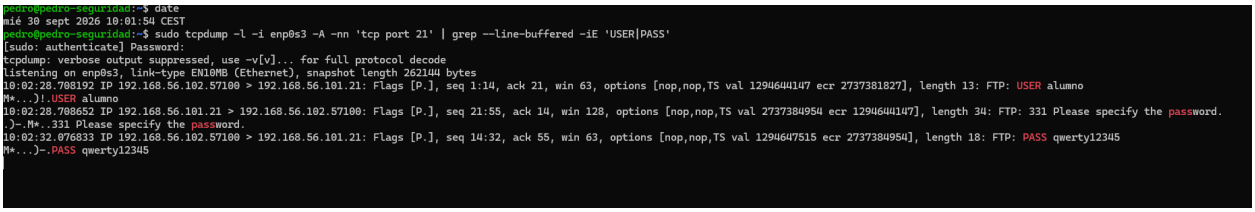
```
ftp 192.168.56.101
```



Inicio de sesión FTP correcto con el usuario alumno.

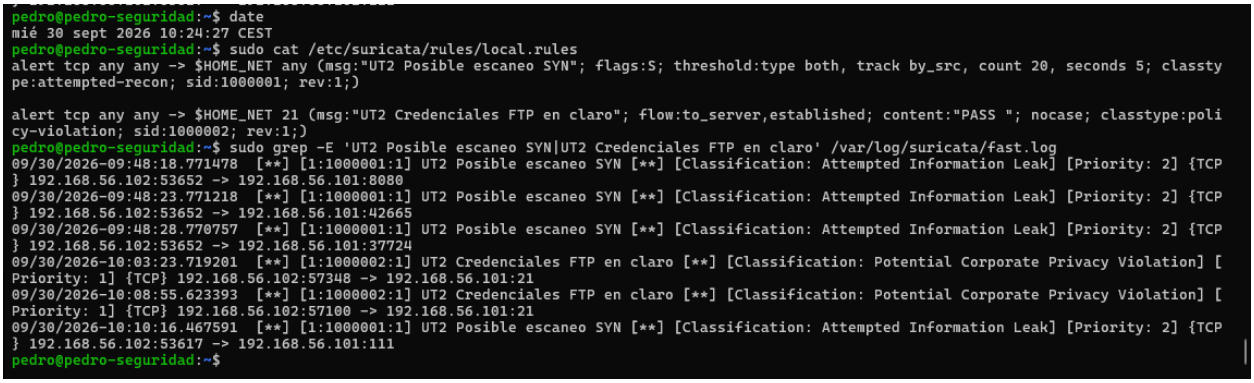
Para comprobar cómo viajaban las credenciales FTP capturé el tráfico del puerto 21 en la interfaz Host-Only `enp0s3`. En la salida aparecieron `USER alumno` y `PASS` seguidos de la contraseña de prueba, así que las credenciales se podían leer directamente en la captura.

```
sudo tcpdump -l -i enp0s3 -A -nn 'tcp port 21' | grep --line-buffered -iE 'USER|PASS'
```



tcpdump muestra los comandos USER y PASS de FTP en texto claro.

La Regla 2 también se activó. En `fast.log` quedó registrada `UT2 Credenciales FTP en claro` con SID 1000002 hacia el puerto 21. Al final comprobé juntas las dos reglas y filtré el registro para dejar visibles únicamente nuestras alertas.

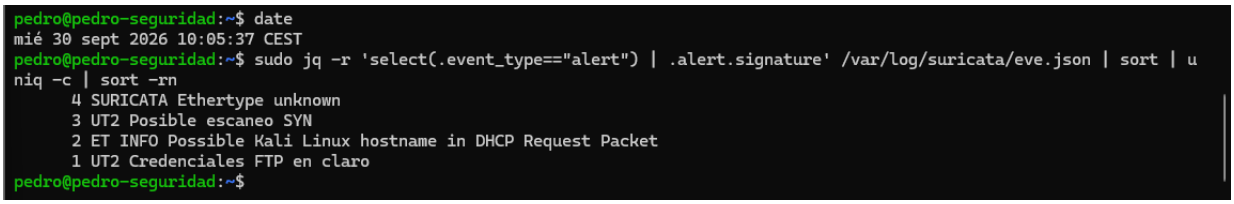


Contenido final de local.rules y alertas de las dos reglas propias filtradas en fast.log.

7.3. Resumen de alertas con jq

También consulté `eve.json` con `jq` para resumir cuántas veces había aparecido cada firma. El resultado real fue: 4 alertas `SURICATA Ethertype unknown`, 3 `UT2 Posible escaneo SYN`, 2 `ET INFO Possible Kali Linux hostname in DHCP Request Packet` y 1 `UT2 Credenciales FTP en claro` en ese momento de la práctica.

```
sudo jq -r 'select(.event_type=="alert") | .alert.signature' /var/log/suricata/eve.json |
sort | uniq -c | sort -rn
```



Resumen de firmas y número de alertas obtenido desde eve.json con jq.

8. Paso 6 - Contramedida y reducción de la superficie de ataque

Después de comprobar las alertas, desactivé `vsftpd`, configuré UFW con política de entrada denegada por defecto y permití únicamente SSH, que necesitaba para seguir administrando el servidor.

```
sudo systemctl disable --now vsftpd
```

```
sudo ufw default deny incoming
```

```
sudo ufw allow 22/tcp
```

```
sudo ufw enable
```

```
sudo ufw status verbose
```

```
pedro@pedro-seguridad:~$ date
mié 30 sept 2026 10:08:00 CEST
pedro@pedro-seguridad:~$ sudo systemctl disable --now vsftpd
[sudo: authenticate] Password:
Synchronizing state of vsftpd.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install disable vsftpd
Removed '/etc/systemd/system/multi-user.target.wants/vsftpd.service'.
pedro@pedro-seguridad:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
pedro@pedro-seguridad:~$ sudo ufw allow 22/tcp
Rules updated
Rules updated (v6)
pedro@pedro-seguridad:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
pedro@pedro-seguridad:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)

pedro@pedro-seguridad:~$
```

vsftpd deshabilitado y UFW activo con deny incoming y excepción para SSH por 22/tcp.

Para comprobar el cambio repetí `nmap -sV` desde Kali. Antes de la contramedida estaban abiertos FTP y SSH. Después, el puerto 21 dejó de aparecer abierto y únicamente quedó accesible el puerto 22 de SSH; los demás puertos aparecieron filtrados por UFW.

```
nmap -sV 192.168.56.101
```

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV 192.168.56.101
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-30 10:10 +0200
Nmap scan report for 192.168.56.101
Host is up (0.00075s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 10.2p1 Ubuntu 2ubuntu3.6 (Ubuntu Linux; protocol 2.0)
MAC Address: 08:00:27:C8:00:F9 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 5.49 seconds

┌──(kali㉿kali)-[~]
└─$ date
Wed Sep 30 10:10:46 AM CEST 2026
```

Nmap después de la contramedida: solo queda accesible SSH por el puerto 22.

El escaneo final volvió a generar una alerta de la Regla 1. Con FTP ya cerrado, Suricata seguía detectando el escaneo mientras únicamente SSH permanecía accesible.

9. Cuestionario

Respondo las preguntas a partir de lo observado en el laboratorio y de la teoría de la Unidad 2.

1. Si cambiaras el umbral de la Regla 1 de count 20 a count 2, ¿qué pasaría con el número de alertas? Relaciónalo con «el cálculo de los falsos positivos que hunde los proyectos» (2.5): ¿por qué un umbral demasiado bajo hace menos segura la red y no más?

El número de alertas aumentaría mucho porque bastarían 2 SYN en 5 segundos para disparar la regla. Ese comportamiento también puede aparecer en tráfico normal, así que crecerían los falsos positivos. Si el sistema genera más avisos de los que se pueden revisar, se acaba perdiendo tiempo con ruido y una alerta real puede pasar desapercibida. Por eso un umbral más bajo no significa automáticamente más seguridad: hay que ajustarlo para detectar el comportamiento sospechoso sin saturar al administrador.

2. La Regla 2 no detectaría nada si la empresa usara SFTP en lugar de FTP. ¿Por qué? ¿Qué principio de la tríada CIA protege el cifrado del transporte?

La Regla 2 busca literalmente el comando `PASS ` dentro del tráfico FTP. Con SFTP la sesión va cifrada sobre SSH, por lo que Suricata no vería la contraseña como texto legible dentro de los paquetes y esa firma no coincidiría. El principio de la tríada CIA que protege el cifrado del transporte es la confidencialidad, porque evita que un tercero pueda leer la información capturada en la red.

3. ¿Qué tendrías que cambiar para que Suricata bloquee el ataque en vez de solo avisar (pasar de IDS a IPS)? Nombra la acción de la regla y el modo en que debe correr Suricata.

Tendría que ejecutar Suricata en modo IPS, en línea con el tráfico, en lugar de usarlo solo como IDS. En la regla cambiaría la acción `alert` por una acción de bloqueo como `drop` (o `reject` si se quiere cortar la conexión de forma explícita). En modo IDS `alert` solo registra el evento; en modo IPS Suricata tiene que estar situado en el camino del tráfico para poder descartar los paquetes.

4. Sitúa cada una de tus dos reglas en la tabla de fases del ataque (1.8): ¿en qué fase actúa la del escaneo SYN y en qué fase la de las credenciales FTP?

La Regla 1 actúa en la fase de reconocimiento, porque detecta el escaneo con el que se buscan puertos y servicios antes de intentar entrar. La Regla 2 encaja con la fase de entrega y acceso inicial, ya que observa el uso de una credencial durante el acceso a un servicio FTP. En nuestra prueba, la contraseña se podía ver en claro, así que también afectaba a la confidencialidad.

5. El apartado 2.5 pide dimensionar el almacenamiento. Si tu Suricata generase 500 eventos/segundo y cada evento ocupara 500 bytes, ¿cuántos GB al día produciría? ¿Y con 30 días de retención? (Muestra el cálculo.)

500 eventos/s × 500 bytes = 250.000 bytes/s. En un día: 250.000 × 86.400 = 21.600.000.000 bytes, que son aproximadamente 21,6 GB/día usando GB decimales. Para 30 días: 21,6 × 30 = 648 GB. Este cálculo no incluye el espacio adicional que puedan necesitar índices, réplicas u otros metadatos.

6. ¿Por qué se recomienda ejecutar siempre suricata -T antes de reiniciar el servicio? ¿Qué relación tiene con «probar en pre-producción antes de aplicar» de la gestión de parches (1.4)?

Porque `suricata -T` comprueba la configuración y las reglas sin arrancar el servicio con los cambios. Si hay una ruta incorrecta o un error de sintaxis, se detecta antes de reiniciar y se evita dejar el IDS caído. La relación con probar los parches en pre-producción es la misma idea: validar el cambio en un entorno de prueba antes de aplicarlo al servicio real para reducir el riesgo de provocar una incidencia.

10. Resultado final

Al terminar la práctica, Suricata quedó funcionando como IDS sobre la interfaz Host-Only. Las dos reglas propias se validaron y generaron alertas durante las pruebas: una con el escaneo SYN y otra al enviar las credenciales por FTP. Después deshabilité FTP y activé UFW dejando únicamente SSH accesible. Al repetir Nmap, FTP ya no aparecía abierto y Suricata seguía registrando el escaneo.

Durante la práctica tuve dos incidencias principales: Ubuntu se quedó sin salida a Internet al trabajar solo con Host-Only y Suricata no encontraba `local.rules` en la ruta esperada. Las dos están documentadas con las comprobaciones y cambios que hice hasta resolverlas.

Fuentes utilizadas

• Detectar un ataque con Suricata: del paquete a la regla, Unidad 2 de Seguridad y Alta Disponibilidad.

• Unidad 2: Seguridad activa: ataques y contramedidas, apartados 1.1, 1.4, 1.8, 2.3 y 2.5.